Guia de Estudo

Data Analysis com Python

Questões orientadas à pesquisa, comparação e experimentação

Este guia cobre a matéria avaliada na certificação *Data Analysis*, organizada pelos seis temas da prova. Para cada questão:

- pesquisa o conceito;
- escreve e executa um pequeno teste em Python;
- compara com casos parecidos;
- explica, por palavras tuas, as diferenças que observaste.

As questões marcadas com **Pesquisa autónoma** tratam temas que não foram trabalhados em detalhe nas aulas.

Cegid Academy

Conteúdo

1 Python para Análise de Dados I **2**
1.1 Estruturas de dados e sintaxe . . . 2
1.2 Tipos, conversões e operadores . . . 2
1.3 Controlo de fluxo . . . 3
1.4 Funções, módulos e bibliotecas . . . 3
1.5 Entrada, saída e comentários . . . 4
1.6 Listas e funções embutidas . . . 4

2 Python para Análise de Dados II **5**
2.1 Dicionários, conjuntos e exceções . . . 5
2.2 Ficheiros . . . 5
2.3 Compreensões de listas . . . 5
2.4 NumPy e pandas — primeiros contactos . . . 6

3 Python para Análise de Dados III **7**
3.1 Estruturas: mutabilidade, referências e cópias . . . 7
3.2 Funções anónimas e avaliação lógica . . . 7
3.3 Módulos e bibliotecas externas . . . 7
3.4 Estrutura de dados em pandas . . . 8
3.5 Transformação e análise . . . 8
3.6 NumPy — médias em matrizes . . . 9

4 Modelos Analíticos com Python I **9**
4.1 Ecossistema de bibliotecas . . . 9
4.2 Leitura e inspeção de dados . . . 9
4.3 Seleção, filtragem e ordenação . . . 10
4.4 Valores em falta . . . 10
4.5 NumPy . . . 10
4.6 Visualização . . . 10

5 Modelos Analíticos com Python II **11**
5.1 Datas e criação de colunas . . . 11
5.2 Índice e filtragem . . . 11
5.3 Agregação, junção e contagens . . . 12
5.4 Estatística e outliers . . . 12
5.5 NumPy: dispersão, aleatoriedade e matrizes . . . 13

6 Modelos Analíticos com Python III **13**
6.1 Operações por grupo . . . 13
6.2 Ordenação, rankings e amostragem . . . 14
6.3 Aplicar funções e alinhar dados . . . 14
6.4 Estatística e memória . . . 15
6.5 Álgebra linear com NumPy . . . 15
6.6 Dimensionalidade e multicolinearidade . . . 16
6.7 Exportação de gráficos e consolidação . . . 16

Modelo de resposta para os alunos **16**

1. Python para Análise de Dados I

1.1. Estruturas de dados e sintaxe

Questão 1.1
Pesquisa quais são os delimitadores usados em Python para criar listas, tuplos, dicionários e conjuntos. Cria um exemplo de cada estrutura, aplica `type()` a todas e explica como se distingue cada uma apenas pela sintaxe. Verifica também o que é criado por um par de chavetas vazio.

Questão 1.2
Investiga a sintaxe correta para criar um dicionário com pares chave–valor. Experimenta escrever os mesmos pares com outros delimitadores e regista quais as versões que produzem erro e porquê.

Questão 1.3
Explora a diferença entre lista e dicionário: como se acede a um elemento em cada um? Implementa um exemplo de cada e explica em que situações cada estrutura faz mais sentido.

1.2. Tipos, conversões e operadores

Questão 1.4
Pesquisa o que devolve a função `type()`. Aplica-a a vários valores literais (inteiro, número decimal, texto, booleano, `None`) e a valores dentro de estruturas. Explica a diferença entre o *valor* e o *tipo* de um objeto e porque é que o resultado aparece no formato `<class '...'>`.

Questão 1.5
Investiga as funções de conversão `int()`, `float()` e `str()`. Converte texto que representa números (inteiros e decimais) e testa o que acontece com texto que não representa um número.

Questão 1.6
Pesquisa os operadores aritméticos `+`, `-`, `*`, `/`, `//`, `%` e `**`. Testa-os com pares de números inteiros e regista, para cada caso, o valor e o *tipo* do resultado. Que operador devolve sempre um número decimal, mesmo quando a divisão é exata?

Questão 1.7
Pesquisa os operadores de comparação em Python (igualdade, diferença, maior, menor...). Compara o operador de igualdade com o operador de atribuição e com o

operador de diferença.

Questão 1.8
Explora as operações com *strings*: junta textos com `+`, repete um texto com `*` e usa um ciclo para imprimir uma linha cada vez maior (por exemplo, um carácter repetido um número crescente de vezes). O que acontece quando tentas somar texto e um número?

Questão 1.9
Pesquisa os operadores de atribuição composta (`+=`, `-=`, `*=`...). Implementa exemplos com números e com *strings* e compara com a forma longa equivalente.

1.3. Controlo de fluxo

Questão 1.10
Investiga as instruções condicionais do Python. Implementa exemplos com `if`, `if...else` e `if...elif...else`, e explica o papel da indentação.

Questão 1.11
Pesquisa a estrutura de repetição controlada por uma condição (e não por uma sequência de elementos). Implementa um exemplo, explica quando termina e o que acontece se a condição nunca deixar de ser verdadeira.

Questão 1.12
Compara os ciclos `for` e `while`. Implementa o mesmo problema com os dois e explica em que situações cada um é mais adequado. Usa também `for` para percorrer uma *string* e uma lista.

Questão 1.13
Pesquisa o que fazem as instruções `break`, `continue` e `pass` dentro de um ciclo. Implementa um único exemplo que use as três e descreve o efeito de cada uma.

Questão 1.14
Explora a função `range()` com um, dois e três argumentos. Para cada caso, converte o resultado em lista e indica o primeiro valor, o último valor e quantos números são gerados. Qual a forma mais simples de obter os inteiros de 0 a $n-1$?

1.4. Funções, módulos e bibliotecas

Questão 1.15
Pesquisa como se define corretamente uma função em Python. Implementa uma função sem argumentos que escreva uma mensagem e outra com argumentos que

devolva um valor. Identifica os elementos obrigatórios da definição e compara com a sintaxe de outras linguagens que conheças.

Questão 1.16
Investiga as formas de importar código em Python: `import modulo`, `import modulo as alias` e `from modulo import nome`. Verifica se as palavras usadas para o mesmo efeito noutras linguagens que conheças funcionam em Python.

1.5. Entrada, saída e comentários

Questão 1.17
Pesquisa qual é a função usada para ler texto escrito pelo utilizador no teclado. Testa o tipo do valor devolvido quando o utilizador escreve um número e mostra como o converter.

Questão 1.18
Explora as diferentes formas de mostrar dados rapidamente durante uma análise, num *script* e num *notebook*: como texto, como tabela formatada e como gráfico. Compara o que cada forma faz e em que contexto se usa.

Questão 1.19
Pesquisa qual é o símbolo usado para comentários de uma linha em Python e como se escrevem comentários de várias linhas (*docstrings*). Compara com os símbolos de comentário de outras linguagens que conheças.

1.6. Listas e funções embutidas

Questão 1.20
Investiga os métodos para acrescentar elementos a uma lista: `append()`, `insert()` e `extend()`. O que acontece quando o elemento acrescentado com `append()` é ele próprio uma lista? Compara com `extend()`.

Questão 1.21
Pesquisa como remover elementos de uma lista: por valor (`remove()`) e por posição (`pop()`, `del`). Testa o que acontece quando o valor aparece repetido e quando não existe na lista.

Questão 1.22
Pesquisa como saber o tamanho de uma coleção em Python. Testa a mesma função com *strings*, tuplos, dicionários e conjuntos. Compara com o método `count()`.

Questão 1.23
Investiga as funções embutidas `abs()`, `sum()`, `min()`, `max()` e `round()`. Testa com números positivos, negativos e com listas numéricas, e interpreta os resultados.

2. Python para Análise de Dados II

2.1. Dicionários, conjuntos e exceções

Questão 2.1
Pesquisa a diferença entre aceder a um valor de um dicionário com `d[chave]` e com o método `get()`. Testa ambos com chaves que existem e que não existem, usando e não usando o segundo argumento de `get()`. O `get()` altera o dicionário?

Questão 2.2
Investiga o que acontece quando passas para `set()` uma lista com elementos repetidos. Que tipo de objeto é devolvido e como aparece escrito? A ordem dos elementos é garantida?

Questão 2.3
Pesquisa as operações entre conjuntos (união, interseção, diferença e diferença simétrica), tanto com operadores como com métodos. Verifica também se os operadores aritméticos funcionam entre conjuntos.

Questão 2.4
Pesquisa a sintaxe de tratamento de exceções em Python. Implementa um exemplo que capture um erro específico, outro que capture qualquer exceção e guarde a mensagem numa variável, e explica para que servem `else` e `finally`.

2.2. Ficheiros

Questão 2.5
Investiga os modos de abertura de ficheiros com `open()`: leitura, escrita, acrescento e criação exclusiva. Para cada modo, indica a letra usada e o que acontece se o ficheiro já existir ou não existir.

Questão 2.6
Pesquisa a diferença entre abrir um ficheiro com `open()` e fechá-lo manualmente e usar `with open(...) as f`. Explica a vantagem da segunda forma.

2.3. Compreensões de listas

Questão 2.7
Investiga as *list comprehensions*. Cria, a partir de `range()`, uma lista de valores transformados (por exemplo, potências) e compara com o ciclo `for` equivalente. Cria também uma versão com condição.

2.4. NumPy e pandas — primeiros contactos

Questão 2.8
Pesquisa como converter uma lista Python num *array* NumPy. Compara o tipo obtido com o de uma lista e o resultado de multiplicar ambos por 2.

Questão 2.9
Investiga as funções do NumPy para criar *arrays* já preenchidos: `np.zeros()`, `np.ones()`, `np.full()` e `np.empty()`. Testa-as com um número e com um tuplo de dimensões.

Questão 2.10
Pesquisa o tipo de objeto devolvido quando se seleciona uma coluna de um DataFrame com `df['coluna']` e quando se selecionam várias colunas com `df[['c1', 'c2']]`. Compara também com a seleção por atributo (`df.coluna`).

Questão 2.11
Explora os métodos do pandas para lidar com valores em falta: `isna()`, `dropna()` e `fillna()`. Num pequeno DataFrame com `NaN`, compara remover linhas com substituir os valores (por zero ou por outro valor).

Questão 2.12
Pesquisa o que fazem `df.head()` e `df.tail()`. Quantas linhas mostram por omissão? Como se altera esse número?

Questão 2.13
Investiga como calcular a média de *uma coluna* de um DataFrame e compara com a média de *todo* o DataFrame. Calcula também soma, mediana, máximo e desvio-padrão.

Questão 2.14
Pesquisa como filtrar as linhas de um DataFrame com uma condição sobre uma coluna (por exemplo, valores acima de um limite). Testa combinações de condições e explica porque é preciso usar `&`, `|` e parênteses.

3. Python para Análise de Dados III

3.1. Estruturas: mutabilidade, referências e cópias

Questão 3.1
Pesquisa o que significa um objeto ser *mutável* ou *imutável*. Tenta alterar um elemento de uma lista e de um tuplo e regista o resultado. Indica qual das duas estruturas é mutável e dá um exemplo de situação em que um tuplo é preferível.

Questão 3.2
Investiga a diferença entre *criar uma nova referência* para um dicionário (atribuição simples) e *criar uma cópia*. Testa várias formas de obter um segundo dicionário a partir de um existente (atribuição, método `copy()`, função `dict()` e desempacotamento com `**`), altera o segundo e verifica se o original muda. Usa `is` para confirmar.

Questão 3.3
Investiga as funções `sorted()` e o método `sort()`. Compara o que cada uma devolve e o que altera. Aplica `sorted()` a uma lista, a um tuplo, a um conjunto e a uma *string*, com e sem `reverse=True`, e indica sempre o tipo do resultado.

3.2. Funções anónimas e avaliação lógica

Questão 3.4
Pesquisa as funções `lambda`, `map()` e `filter()`. Usa `map()` para transformar todos os elementos de uma lista e `filter()` para ficar só com os que cumprem uma condição. Converte os resultados em lista e compara com *list comprehensions* equivalentes. Que tipo de objeto devolvem `map()` e `filter()` antes da conversão?

Questão 3.5
Pesquisa que valores o Python considera "falsos" num contexto lógico (por exemplo, números zero, texto vazio, listas vazias, `None`). Testa `bool()` com cada um. Depois explora as funções `any()` e `all()` com listas que misturam valores verdadeiros e falsos.

3.3. Módulos e bibliotecas externas

Questão 3.6
Pesquisa o que é um módulo em Python. Cria um ficheiro com algumas funções, importa-o a partir de outro ficheiro ou *notebook* na mesma pasta e usa as funções. Que extensão tem de ter o ficheiro?

Questão 3.7

Investiga a diferença entre *instalar* e *importar* uma biblioteca. Que bibliotecas já vêm com o Python e quais é preciso instalar? Pesquisa como se instala uma biblioteca externa com `pip` (num terminal e num *notebook*) e como se verifica a versão instalada.

3.4. Estrutura de dados em pandas

Questão 3.8
Pesquisa qual é a estrutura do pandas mais adequada para representar dados tabulares e compara-a com uma `Series`, uma lista e um dicionário. Indica quantas dimensões tem cada uma.

Questão 3.9
Investiga como obter os nomes das colunas de um DataFrame. Que tipo de objeto devolve o atributo `columns`? Como se converte esse resultado numa lista Python simples? Escreve o resultado esperado antes de executar.

Questão 3.10
Explora o atributo `dtype` de uma coluna e o atributo `dtypes` de um DataFrame. Que tipo aparece para uma coluna de inteiros, de decimais e de texto? Compara com o resultado de `type()` aplicado a um valor da coluna.

3.5. Transformação e análise

Questão 3.11
Explora o método `apply()` aplicado a uma *coluna* (Series): o que acontece a cada elemento? Compara com aplicar `apply()` a um DataFrame e explica o papel do argumento `axis`.

Questão 3.12
Pesquisa o significado do argumento `inplace=True` em métodos do pandas. Testa um método (por exemplo `fillna()` ou `drop()`) com e sem esse argumento e verifica o que acontece ao objeto original e ao valor devolvido.

Questão 3.13
Investiga como contar, coluna a coluna, quantos valores em falta existe num DataFrame. Explica passo a passo o que devolve cada parte da expressão e como obter também a percentagem de valores em falta.

Questão 3.14
Explora o método `groupby()`: o que faz e que tipo de resultado produz? Implementa um exemplo e descreve o padrão dividir–aplicar–combinar. Compara com filtrar, ordenar ou remover duplicados.

Questão 3.15
Pesquisa como relacionar duas tabelas do pandas através de uma coluna-chave partilhada (o equivalente a um *join* em bases de dados). Compara a função `pd.merge()` com o método `df.join()` e com `pd.concat()`. Em que situação se usa cada uma?

Questão 3.16
Investiga como medir a associação linear entre as variáveis numéricas de um DataFrame. Que estrutura devolve o método? Interpreta valores próximos de 1, de -1 e de 0, e compara com o método que calcula a covariância.

Questão 3.17
Explora a criação de gráficos diretamente a partir do pandas: `df.plot()`, `df['coluna'].plot()` e o acessor `df.plot.<tipo>()`. Gera um gráfico de linhas a partir de uma única coluna de várias formas e compara com a abordagem usando diretamente Matplotlib.

3.6. NumPy — médias em matrizes

Questão 3.18
Investiga o efeito do argumento `axis` em `np.mean()` aplicado a uma matriz 2D (por exemplo, uma lista de listas). Calcula a média sem `axis`, com `axis=0` e com `axis=1` e compara a forma dos resultados.

4. Modelos Analíticos com Python I

4.1. Ecossistema de bibliotecas

Questão 4.1
Pesquisa para que serve cada uma das principais bibliotecas de análise de dados em Python: NumPy, pandas, Matplotlib, Seaborn e scikit-learn. Indica, para cada uma, o tipo de tarefa em que é a escolha habitual e a forma convencional de a importar (incluindo o módulo `pyplot` do Matplotlib).

4.2. Leitura e inspeção de dados

Questão 4.2
Pesquisa as funções do pandas para ler e escrever ficheiros: CSV, Excel e texto delimitado. Distingue as funções de *leitura* das de *escrita* e testa parâmetros como o separador e o cabeçalho na leitura de um CSV.

Questão 4.3
Explora os métodos de inspeção inicial de um DataFrame: `head()`, `info()` e `describe()`. Que informação dá cada um (linhas, tipos, valores não nulos, estatísticas)? Que tipo de objeto devolvem `head()` e `describe()`?

4.3. Seleção, filtragem e ordenação

Questão 4.4
Pesquisa como selecionar uma coluna, várias colunas e um intervalo de linhas de um DataFrame. Compara `loc` e `iloc`: o que usa cada um para selecionar? Testa a fatia `0:3` com ambos e explica porque o número de linhas obtido pode ser diferente.

Questão 4.5
Revê a filtragem de linhas com condições: seleciona as linhas em que uma coluna numérica é superior a um valor e conta quantas são. Explica porque é que a condição tem de ser escrita sobre a coluna e não sobre o nome da coluna.

Questão 4.6
Pesquisa como ordenar um DataFrame pelos valores de uma coluna, por ordem crescente e decrescente. Compara com a ordenação pelo índice. Que método e que argumento controlam a ordem?

4.4. Valores em falta

Questão 4.7
Investiga diferentes estratégias para lidar com valores em falta: remover linhas, substituir por um valor fixo, substituir por uma estatística (média, mediana) e propagar o valor anterior. Implementa cada uma e compara com o método `replace()`.

4.5. NumPy

Questão 4.8
Pesquisa as funções do NumPy para gerar sequências de números: `np.arange()` e `np.linspace()`. Gera com cada uma os valores entre 0 e 1 e explica a diferença entre indicar o *passo* e indicar o *número de valores*. Qual delas inclui o valor final?

Questão 4.9
Explora as funções estatísticas do NumPy (`np.mean()`, `np.median()`, `np.sum()`, `np.std()`) e os métodos equivalentes do pandas para uma Series (`mean()`, `median()`...). Aplica-os aos mesmos dados e compara.

4.6. Visualização

Questão 4.10
Pesquisa as funções básicas do `matplotlib.pyplot`: gráfico de linhas, de dispersão, de barras e histograma. Qual a função mais usada para desenhar linhas? Acrescenta título, legendas dos eixos e mostra a figura.

Questão 4.11
Explora a diferença entre Matplotlib e Seaborn. Em que tipo de gráficos se especializa cada uma? Escolhe um gráfico (por exemplo, distribuição de uma variável por categoria) e implementa-o com as duas bibliotecas.

5. Modelos Analíticos com Python II

5.1. Datas e criação de colunas

Questão 5.1 Pesquisa autónoma
Pesquisa como converter uma coluna de texto com datas para o tipo `datetime` no pandas. Verifica o `dtype` antes e depois da conversão e usa o acessor `.dt` para extrair ano, mês e dia da semana. O que acontece se uma das datas estiver mal escrita?

Questão 5.2 Pesquisa autónoma
Investiga como criar uma nova coluna a partir de operações entre colunas existentes (soma, diferença, produto). Compara a forma vetorizada com um ciclo e com o método `sum(axis=1)`.

Questão 5.3 Pesquisa autónoma
Pesquisa como criar uma coluna booleana a partir de uma condição lógica sobre outra coluna. Depois investiga `np.where()` para atribuir rótulos de texto em vez de `True`/`False`. Compara com o método `where()` do pandas.

Questão 5.4
Explora como executar uma função tua sobre os dados de um DataFrame, registo a registo. Que método e que valor de `axis` são necessários? Compara com os métodos `map()`, `transform()` e `filter()` e indica a que se aplica cada um.

Questão 5.5
Investiga a normalização Min–Max. Escreve a fórmula e implementa-a manualmente numa coluna de um DataFrame. Compara com a padronização Z-score: em que intervalo ficam os valores em cada caso?

5.2. Índice e filtragem

Questão 5.6 **Pesquisa autónoma**

Pesquisa o método `reset_index()` e explica quando é útil. Aplica-o depois de filtrar ou ordenar um DataFrame e observa o índice. Compara `drop=True` com `drop=False` e compara com `set_index()` e `reindex()`.

Questão 5.7

Compara várias formas de selecionar linhas que cumprem uma condição: indexação booleana, `query()` e `where()`. Para cada uma, verifica a sintaxe correta, o número de linhas do resultado e se aparecem valores `NaN`.

5.3. Agregação, junção e contagens

Questão 5.8

Pesquisa como calcular uma estatística por grupo (por exemplo, a média de uma coluna numérica para cada categoria). Explica o papel de cada parte da expressão (coluna de agrupamento, coluna a agregar, função) e o tipo de objeto devolvido.

Questão 5.9

Pesquisa como construir uma tabela dinâmica em pandas com `pivot_table()`: que argumentos definem as linhas, as colunas, os valores e a função de agregação? Compara com `pivot()` (o que acontece quando há combinações repetidas?) e com `groupby()` + `unstack()`.

Questão 5.10

Investiga as diferenças entre junções *inner*, *left*, *right* e *outer* com `pd.merge()`. Implementa um exemplo com duas tabelas e regista que linhas ficam em cada caso. Pesquisa também os parâmetros `on`, `left_on`/`right_on` e qual é o tipo de junção por omissão.

Questão 5.11

Pesquisa as diferentes formas de contar numa coluna: `count()`, `unique()`, `nunique()` e `value_counts()`. Aplica as quatro à mesma coluna categórica e descreve exatamente o que cada uma devolve (um número? uma lista de valores? uma contagem por valor?).

Questão 5.12

Pesquisa como ordenar um DataFrame por *várias* colunas ao mesmo tempo, incluindo ordens diferentes para cada coluna. Compara com outros métodos de ordenação ou classificação (`sort_index()`, `rank()`).

5.4. Estatística e outliers

Questão 5.13 **Pesquisa autónoma**
Investiga a diferença entre calcular a matriz de correlação de um DataFrame e calcular a correlação entre *duas* colunas específicas. Que método se chama sobre uma coluna e que argumento recebe? Implementa ambos num DataFrame com três colunas numéricas.

Questão 5.14
Pesquisa como calcular os quartis Q1 e Q3 de uma coluna numérica e o intervalo interquartil (IQR). Que método do pandas se usa e em que escala se indica o quantil (percentagem ou proporção)? Compara com usar a média ou a mediana.

Questão 5.15
Pesquisa diferentes abordagens para tratar *outliers* antes de aplicar um modelo analítico: remover, limitar (*capping*), transformar ou manter. Discute vantagens e riscos de cada uma e explica porque não se deve remover ou substituir valores extremos sem análise.

5.5. NumPy: dispersão, aleatoriedade e matrizes

Questão 5.16 **Pesquisa autónoma**
Pesquisa a função do NumPy que calcula o desvio-padrão e compara-a com a que calcula a variância. Aplica `np.std()` e o método `std()` do pandas aos mesmos números e verifica se coincidem. Investiga o parâmetro `ddof`.

Questão 5.17 **Pesquisa autónoma**
Investiga o módulo `np.random`: compara `normal()`, `uniform()`, `rand()` e `choice()`. Gera 1000 valores de uma distribuição normal com média 0 e desvio-padrão 1, verifica a média e o desvio obtidos e explica para que serve fixar a semente.

Questão 5.18 **Pesquisa autónoma**
Pesquisa as formas de calcular a transposta de uma matriz em NumPy (atributo, método e função). Implementa uma matriz 2×3 e verifica se todas produzem o mesmo resultado e a mesma forma.

6. Modelos Analíticos com Python III

6.1. Operações por grupo

Questão 6.1
Explora a diferença entre `agg()`, `transform()`, `apply()` e `filter()` aplicados a dados agrupados com `groupby()`. Cria um DataFrame com grupos, aplica a média

com cada método e compara o *formato* do resultado (número de linhas e índice). Qual devolve um resultado com o mesmo número de linhas do DataFrame original?

Questão 6.2
Pesquisa como calcular *várias* estatísticas ao mesmo tempo para cada grupo (por exemplo média e soma) com `groupby()` e `agg()`. Obtém o mesmo resultado com `pivot_table()` usando uma lista de funções em `aggfunc`. Explica a ordem dos elementos: coluna de agrupamento, coluna a agregar, funções.

Questão 6.3 **Pesquisa autónoma**
Investiga as janelas móveis com `rolling(n)`. Cria uma série de valores ao longo do tempo e calcula a média móvel com uma janela de 3 períodos. Explica o que acontece aos primeiros valores e porquê.

6.2. Ordenação, rankings e amostragem

Questão 6.4 **Pesquisa autónoma**
Pesquisa o método `rank()` e o parâmetro `method` (`'average'`, `'min'`, `'first'`, `'dense'`). Cria uma coluna com valores repetidos e compara os rankings obtidos. Como se obtém um ranking decrescente (o maior valor em 1.º lugar) sem empates? Guarda o resultado numa nova coluna.

Questão 6.5 **Pesquisa autónoma**
Pesquisa o método `nlargest(n)` (e `nsmallest(n)`). Usa-o para obter os três maiores valores de uma coluna e compara com ordenar e ficar com as primeiras linhas. O que acontece quando há valores repetidos? Como garantirias que obténs valores *distintos*?

Questão 6.6 **Pesquisa autónoma**
Investiga como obter uma amostra aleatória das linhas de um DataFrame, indicando um número fixo de linhas ou uma *proporção*. Que argumento se usa em cada caso? Pesquisa também o parâmetro que torna a amostra reprodutível.

6.3. Aplicar funções e alinhar dados

Questão 6.7 **Pesquisa autónoma**
Compara três formas de aplicar uma função num DataFrame: `apply()` (por coluna ou por linha), `map()` numa Series e a aplicação *elemento a elemento* a todo o DataFrame. Pesquisa como se chama o método para este último caso nas versões antigas e nas recentes do pandas. E se a função precisar de vários valores do mesmo registo?

Questão 6.8 **Pesquisa autónoma**
Pesquisa o alinhamento automático de dados por índice no pandas. Cria dois Data-

Frames (ou Series) com índices só parcialmente iguais, soma-os diretamente e observa o resultado. Depois investiga o método `align()` e compara com `merge()`, `join()` e `concat()`.

6.4. Estatística e memória

Questão 6.9 **Pesquisa autónoma**

Pesquisa como medir a memória ocupada por um DataFrame, coluna a coluna. Compara o resultado com e sem o argumento `deep=True` num DataFrame com colunas de texto e explica a diferença. Compara com a informação de memória apresentada por `info()`.

Questão 6.10 **Pesquisa autónoma**

Investiga `df.cov()` e compara com `df.corr()`, `df.var()` e `df.describe()`. Aplica-os ao mesmo DataFrame numérico e explica o que devolve cada um e a diferença de interpretação entre covariância e correlação.

Questão 6.11

Investiga as medidas de dispersão disponíveis num DataFrame e como obtê-las para todas as colunas numéricas de uma só vez. Compara com a média, a variância e o resultado de `describe()`: qual devolve *apenas* o desvio-padrão?

Questão 6.12

Revê a regra do IQR para identificar *outliers*: quais são os limites inferior e superior? Implementa-a numa coluna e compara com outros critérios de deteção de valores extremos que encontres na tua pesquisa.

6.5. Álgebra linear com NumPy

Questão 6.13 **Pesquisa autónoma**

Pesquisa o submódulo `np.linalg` e a função que calcula a inversa de uma matriz quadrada. Implementa um exemplo e verifica o resultado multiplicando a matriz pela inversa (o produto deve ser a matriz identidade). O que acontece com uma matriz singular?

Questão 6.14 **Pesquisa autónoma**

Investiga como resolver um sistema de equações lineares $Ax = b$ com NumPy. Representa um sistema de 2 equações em forma matricial e resolve-o de duas formas: com a função própria para sistemas e usando a matriz inversa. Compara os resultados e pesquisa qual das formas é recomendada.

Questão 6.15 **Pesquisa autónoma**

Pesquisa como obter os valores próprios (autovalores) e os vetores próprios de uma matriz quadrada com NumPy. Implementa um exemplo simples e interpreta o que é devolvido pela função.

6.6. Dimensionalidade e multicolinearidade

Questão 6.16
Pesquisa o conceito de *multicolinearidade*: o que é, porque é um problema em modelos analíticos (por exemplo, regressão) e como se deteta (matriz de correlação, VIF). Indica técnicas adequadas para lidar com ela.

Questão 6.17
Investiga o conceito de redução de dimensionalidade e o PCA (Análise de Componentes Principais): o que procura preservar e como se implementa em Python. Compara com outras técnicas de pré-processamento que estudaste e indica quais alteram o número de variáveis.

6.7. Exportação de gráficos e consolidação

Questão 6.18
Pesquisa como exportar um gráfico do Matplotlib para um ficheiro. Testa diferentes formatos (PNG, PDF) e valores de `dpi` e compara a resolução e o tamanho dos ficheiros. Porque se deve guardar antes de `plt.show()`?

Questão 6.19
Descreve o fluxo completo de uma análise de dados com pandas: leitura, inspeção, limpeza, tratamento de valores em falta, identificação de *outliers*, normalização, filtragem, agrupamento e visualização. Ilustra cada etapa com uma linha de código, usando um conjunto de dados à tua escolha.

Modelo de resposta para os alunos

Modelo de resposta: Para cada questão, organiza a resposta com a seguinte estrutura:

1. **Conceito pesquisado:** definição curta.
2. **Código de teste:** pequeno exemplo em Python.
3. **Resultado observado:** o que apareceu no ecrã.
4. **Comparação:** diferença face a outro caso parecido.
5. **Conclusão:** o que aprendeste.

```
# Exemplo de resposta (codigo de teste)
```

```
nome = "analise de dados"
print(nome.upper())       # resultado observado: ANALISE DE DADOS
print(nome.title())       # comparacao: Analise De Dados
```

Guia de Estudo — Questões orientadas à pesquisa e experimentação